



Secure Upload Methods in PHP

Here's how to deal with file upload attacks against sites developed in PHP, and how to write more secure code to prevent these attacks.

In most Web applications, developers provide upload file functionality—images, for example. This functionality could be exploited by attackers to upload malicious ‘Web shell’ code, which might give them command-prompt access to the server. In this article, we look at how such upload forms (such as the HTML form code below) can be exploited, and how to prevent such attacks.

```
<form name=upload action=upload.php method=post>
  upload a file: <input type=file name=fileName>
  <input type=submit name=upload>
</form>
```

The simplest implementation

In most implementations, no verification of the uploaded file is done. The form above invokes an *upload.php* PHP script to process the upload; the code in *upload.php* might move the file to a common/well-known folder, without verifying its content—like what's given below:

```
<?php
$uploaddir = 'uploads/'; // Relative path under
```

```
webroot
$uploadfile = $uploaddir . basename($_
FILES['userfile']['name']);
if (move_uploaded_file($_FILES['userfile']['tmp_
name'], $uploadfile)) {
    echo "File is valid, and was successfully
uploaded.\n";
}
else {
    echo "File uploading failed.\n";
}
?>
```

Let's suppose the attacker were to upload a file containing code as follows:

```
<?php
System("ls");
?>
```

In such a case, *upload.php* will move the attackers' file (named *shell.php*, for example) to the *uploads* sub-folder. If, for instance, the attackers then enter *http://site/uploads/shell.php* as the URL in their browser/client, they will get a listing of the current working directory.